

ACKNOWLEDGEMENT

First and foremost, I want to thank everyone that has helped me during this Internship I period. As this would not be possible without them.

I would like to express my deepest gratitude for the internship opportunity granted by **H.E. Mr. Heang Sotheayuth**, Deputy Secretary General of The Council for the Development of Cambodia (CDC). and to the president, **H.E. Dr. Seng Sopheap**, and vice-president, **H.E. Hean Samboeun**, of Cambodia Academy of Digital Technology (CADT) for making it possible for us to participate and study for our bachelor's degree program at CADT.

I want to thank my advisor, ____, for his valuable guidance and advice. Which has helped me a lot in understanding this Internship II program.

I am also very grateful to my supervisor and the project manager, **Mr. Khouch Koeun**, the technical lead **Mr. Van Dara**, and everyone else at the Council for the Development of Cambodia for being very supportive and helpful throughout the internship process, they have shared valuable insights to me whether it is technical or knowledge about the working environment.

I also would like to thank the Department of Computer Science and Career Center at Cambodia Academy of Digital Technology (CADT) for organizing this internship I program and making sure everything goes smoothly.

Finally, I would like to thank my friends and family for their support, especially the friends who also happen to take this internship program at CDC.

មូលន័យសង្ខេប

ក្នុងគម្រោងនេះ ខ្ញុំបានអភិវឌ្ឍ...

ABSTRACT

...

TABLE OF CONTENTS

ACKNOWLEDGEMENT	I
មូលន័យសង្ខេប	II
ABSTRACT	III
TABLE OF CONTENTS	IV
LIST OF FIGURES	VII
LIST OF TABLES	VIII
LIST OF ABBREVIATIONS	IX
I. INTRODUCTION	1
1.1. Presentation of the Internship	1
1.1.1. Objective of Internship	1
1.1.2. Duration of Internship	1
1.2. Presentation of the Organization	2
1.2.1. General Information of the Organization	2
1.2.2. Service of the Organization	2
1.2.3. Vision and Mission	2
1.2.4. Organizational Chart	3
1.2.5. Address and Contact	4
1.3. Project Definition and Scope	4
1.3.1. Problem Statement and Conceptual Rationale	4
1.3.2. Targeted Impact	4
II. Literature Review	5
2.1. Existing Systems Analysis	5
2.1.1. Enterprise Industry Standards: Gmail & Outlook	5
2.1.2. High-Productivity & AI-Driven Clients: Superhuman & Spark	5
2.1.3. Specialized CRM & Contact Integration	5
2.2. Engineering Theory and Design Patterns	6
2.2.1. The Adapter Pattern for Service Abstraction	6
2.2.2. Zero Trust and Context-Aware Security	6
2.2.3. Relational Localization and Data Sovereignty	6
2.2.4. High-Concurrency Data Streaming	6
III. PROJECT ANALYSIS	8
3.1. Requirements Specification	8
3.1.1. Functional Requirements	8

3.1.2. Non-Functional Requirements	9
3.2. Use Case Analysis	10
3.2.1. Account Linking	10
3.2.2. Scanning Business Cards	10
3.2.3. Primary Actor	11
3.2.4. Supporting Actors / External Systems	11
3.2.5. Use Case Diagram (Conceptual)	11
3.3. Process Flow and Activity Concepts	12
3.3.1. Core Workflow (Narrative)	12
3.3.2. Activity Diagram (Conceptual)	12
3.4. Data and Database Design	12
3.4.1. Main Entities	13
3.4.2. Conceptual Entity Relationships	13
3.4.3. Database Design Considerations	13
3.4.4. Sample Data Dictionary (Selected Fields)	14
3.5. Analysis of Technical Architecture	14
3.5.1. Proposed Application Layers	14
3.5.2. API and Data Exchange Considerations	14
IV. DETAIL CONCEPT	16
4.1. Logical Architecture	16
4.1.1. Architecture Style	16
4.1.2. Logical Components	17
4.1.3. Layer Interaction Flow	17
4.1.4. Data Flow Concept	18
4.2. Physical Architecture	18
4.2.1. Deployment Overview	18
4.2.2. Physical Communication Flow	19
4.2.3. Client Device Requirements	19
4.2.4. Security Considerations in Physical Design	19
4.3. Tools and Technology Requirements	20
4.3.1. Technologies	20
a. Flutter (Dart)	20
b. JWT Authentication	20
c. Freezed and json_serializable	20
d. Local Storage / Secure Storage	21

e. OCR Technology	21
f. Notification Service	21
g. Flutter Localization (l10n / ARB)	21
4.3.2. Tools	21
a. Visual Studio Code / Android Studio	21
b. Flutter SDK	22
c. Git	22
d. GitHub / GitLab	22
e. Postman / Insomnia	22
f. Android Emulator / iOS Simulator / Physical Device	22
g. Figma	23
h. Typst	23
i. draw.io / Lucidchart / PlantUML	23
4.3.3. Development Environment Requirements	23
V. IMPLEMENTATION	24
5.1. Project Structures	24
5.2. Sequence Diagram	24
VI. CONCLUSION	25
6.1. Completed Functions	25
6.2. Incomplete Functions	25
6.3. Challenges	25
6.4. Experience	25
6.5. Perspective	25
6.6. Future Work	25
REFERENCES	26
APPENDICES	27

LIST OF FIGURES

Figure 1	The Council for the Development of Cambodia's Logo	2
Figure 2	The CDC's Organization Chart	3
Figure 3	Account Linking Use Case Diagram	10
Figure 4	Business Card Scanning Use Case Diagram	10
Figure 5	Use Case Diagram of cdcIRM (Placeholder)	11
Figure 6	Activity Diagram for Main User Workflow (Placeholder)	12
Figure 7	Database Table (Placeholder)	13
Figure 8	Physical Architecture of the cdcIRM System (Placeholder)	19
Figure 9		20
Figure 10		20
Figure 11		20
Figure 12		21
Figure 13		21
Figure 14		21
Figure 15		21
Figure 16		21
Figure 17		22
Figure 18		22
Figure 19		22
Figure 20		22
Figure 21		22
Figure 22		23
Figure 23		23
Figure 24		23

LIST OF TABLES

Table 1	Functional Requirements of the cdcIRM System	8
Table 2	Non-Functional Requirements	9
Table 3	Selected Data Dictionary	14
Table 4	Logical Components of the cdcIRM Application	17

LIST OF ABBREVIATIONS

Abbreviation	Explanation
CADT	Cambodia Academy of Digital Technology

I. INTRODUCTION

The internship program is a well known bachelor's programs at the Institute of Digital Technology (IDT), Cambodia Academy of Digital Technology (CADT). This program puts students into real world work experience related to their field. This report will cover what I have accomplished and learned during this whole process.

1.1. Presentation of the Internship

I conducted my internship at the Council for the Development of Cambodia (CDC) within the Information Technology and Public Relation Directorate (ITPR). I was tasked with the end-to-end development of a sophisticated mobile email client specifically engineered for government officials to manage investor relations.

The project, titled “cdcIRM”, aims to revolutionize how the CDC interacts with international and local investors. Unlike standard email applications, this system integrates high-productivity tools, such as OCR-based contact scanning and automated follow-up logic to ensure that the CDC remains responsive and efficient in its role as the nation’s “One-Stop Service.”

1.1.1. Objective of Internship

The primary objective of this internship, as outlined by the IDT guidelines, is to offer students golden opportunities to apply the knowledge and skills acquired during their academic studies to solve real-world problems. It provides a platform to experience a professional work environment, acquire new skills, and gain practical professional experience. Through such programs, students can better understand their work preferences, which helps them explore future career options effectively. Furthermore, internships offer significant benefits to host organizations, allowing them to observe and evaluate the student’s technical knowledge, skills, and behavior for potential future employment.

1.1.2. Duration of Internship

The internship commenced on January 5th, 2026, and concluded on July 5th, 2026, spanning a total of six months (24 weeks). In accordance with the CADT Year 4 curriculum requirements, I completed a minimum of 960 hours.

1.2. Presentation of the Organization

This section introduces the organization where I completed my internship. It includes basic information about the organization, its version and mission, and how it is structured. Understanding the role of organization helps explain how my internship project supports its overall goals.

1.2.1. General Information of the Organization



Figure 1: The Council for the Development of Cambodia's Logo

The internship took place at The **Council for the Development of Cambodia (CDC)**, an executive agency of the Royal Government of Cambodia. Established in 1994 under the Law on Foreign Investment in the Kingdom of Cambodia, the CDC functions as the “Etat-Major” and the “One-Stop Service” of the Royal Government. Its responsibilities encompass the rehabilitation and development of Cambodia, overseeing both public and private investments, and facilitating the establishment and management of Special Economic Zones (SEZs).

1.2.2. Service of the Organization

The CDC provides essential services to ensure economic stability and growth:

- Investment Project Management: Evaluating and approving Qualified Investment Projects (QIPs).
- Strategic Coordination: Managing Official Development Assistance (ODA) and partnering with international donors for infrastructure development.
- Special Economic Zones (SEZs): Overseeing the legal and operational frameworks of industrial hubs.
- Investor Aftercare: Providing ongoing support to ensure the success and retention of foreign and local investments.

1.2.3. Vision and Mission

The CDC’s vision is to position Cambodia as a leading destination for high-value investment and sustainable development. Its mission focuses on reducing the “cost of doing business” by cutting through red tape and providing competitive incentives.

To achieve this, the CDC emphasizes Digital Transformation. By adopting “GovTech” solutions, the organization aims to increase transparency and responsiveness. This project—a smart email client—aligns perfectly with this mission by ensuring that official communications are handled with the speed and precision expected by modern global investors.

The CDC’s objectives are broad and strategic. They include attracting targeted international investments, especially in sectors that align with Cambodia’s development needs. It also promotes digital literacy and IT training to prepare the workforce for a tech-driven future. CDC emphasizes centralized development planning, ensuring that all projects support the nation’s broader economic strategies. Additionally, it works to streamline investment procedures for ease and transparency and partners with the Cambodian Rehabilitation and Development Board (CRDB) to manage and coordinate Official Development Assistance (ODA) effectively. Overall, CDC plays a key role in expanding Cambodia’s infrastructure, diversifying the economy, and increasing its competitiveness on the global stage.

1.2.4. Organizational Chart

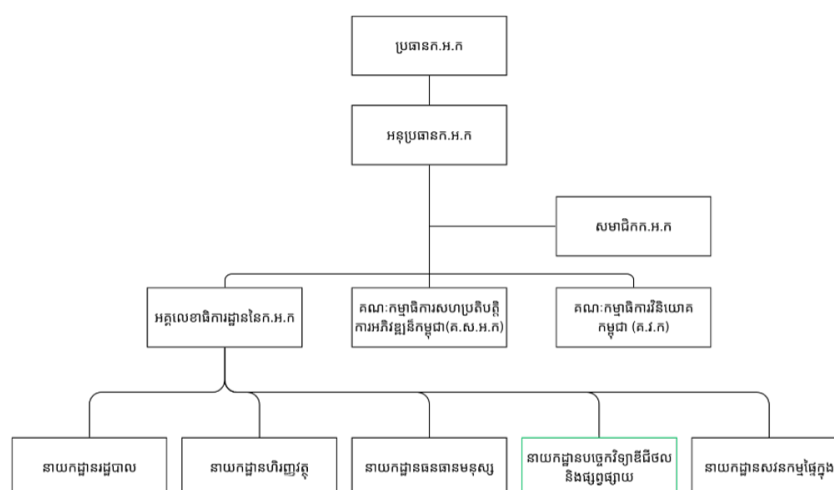


Figure 2: The CDC's Organization Chart

Figure 2 represents the hierarchy of The Council for the Development of Cambodia (CDC). From the top, there is the President of CDC, followed by the Vice-President of CDC. Other mentionable branches are the Cambodia Investment Board (CIB), the Cambodia Special Economic Zone Board (CSEZB), and the General Directorate.

My internship was hosted within the Information Technology and Public Relation Directorate (ITPR), situated under the General Directorate. The ITPR is responsible for the organization’s digital infrastructure, cybersecurity, and public communication channels. My project directly supports the ITPR’s initiative to provide CDC staff with advanced mobile tools that enhance their day-to-day productivity and communication efficiency.

1.2.5. Address and Contact

The Council for the Development of Cambodia is located at:

- Address: [Government Palace, Sisowath Quay, Wat Phnom, Phnom Penh, Cambodia.](#)
- Phone: [\(+855\) 23 981 154](#) / [\(+855\) 23 427 597](#)
- Website: www.cdc.gov.kh
- Email: info@cdc.gov.kh

1.3. Project Definition and Scope

cdcIRM (CDC Investor Relations Management) is a specialized communication platform designed to serve as a digital bridge between the Council for the Development of Cambodia and global investors. While traditional email clients are built for general-purpose correspondence, this project is engineered for “High-Velocity Investor Relations,” providing a focused environment for government-to-business (G2B) interactions.

1.3.1. Problem Statement and Conceptual Rationale

Currently, official correspondence is often fragmented across various platforms, leading to “context-switching” fatigue and potential delays in response times. cdcIRM addresses this by aggregating disparate email services into a single, unified, and intelligent interface. It moves beyond traditional email by integrating productivity tools such as OCR-based contact scanning and automated follow-up logic, ensuring the CDC maintains the precision expected by modern global investors.

The application focuses on three core pillars:

- **Immediacy:** Reducing the time spent on data entry and navigation.
- **Intelligence:** Assisting staff in maintaining relationships through automated reminders.
- **Integration:** Connecting directly with CDC’s backend infrastructure while maintaining government-grade security protocols.

1.3.2. Targeted Impact

By centralizing communication and contact management, cdcIRM aims to revolutionize the CDC’s internal workflows. This project supports the organization’s objective of promoting transparency and national competitiveness on the global stage, ensuring that Cambodia remains an attractive hub for sustainable development.

II. Literature Review

2.1. Existing Systems Analysis

The landscape of email communication has shifted from simple message exchange to complex productivity ecosystems. While traditional clients focus on stability and broad feature sets, modern “power-user” clients prioritize speed and organizational efficiency. This section establishes a benchmark for the cdcIRM application by analyzing leading global solutions.

2.1.1. Enterprise Industry Standards: Gmail & Outlook

- **Gmail:** Recognized for its powerful search capabilities and deep integration with Google Workspace. It pioneered features like “Smart Compose” and tabbed categorization. However, for high-stakes investor relations, users often find its interface “cluttered” and prone to distractions from non-essential automated mail. ^[1]
- **Microsoft Outlook:** The definitive standard for “locked-down” institutional organizations. Its strength lies in robust calendar integration and enterprise-grade security (S/MIME). For the CDC, Outlook is often a primary backend provider; however, its mobile interface is frequently criticized for being heavy and unintuitive for rapid, on-the-go tasks such as contact scanning. ^[2]

2.1.2. High-Productivity & AI-Driven Clients: Superhuman & Spark

- **Superhuman:** Widely considered the fastest email experience currently available. It utilizes a “keyboard-first” philosophy and a “Split Inbox” to separate human-sent emails from automated notifications. A key relevant feature is **AI-powered follow-ups**, which nudge users to maintain momentum in communication—a critical need for investor relations.
- **Spark Mail:** Developed by Readdle, Spark emphasizes team collaboration and “Smart Categorization.” Its standout feature allows users to screen new senders, a valuable concept for government officials who must filter high volumes of unsolicited inquiries.

2.1.3. Specialized CRM & Contact Integration

Platforms such as **Salesforce Anywhere** and **HubSpot Mobile** facilitate lead management by allowing users to scan business cards to create CRM entries. While these provided the inspiration for the cdcIRM contact scanning module, they are proprietary, high-cost platforms

that do not natively integrate with the specific ID formats or the private server infrastructure required by the Cambodian government.

2.2. Engineering Theory and Design Patterns

To bridge the gaps identified in existing systems, this project utilizes several computer science patterns to ensure scalability, security, and bilingual support.

2.2.1. The Adapter Pattern for Service Abstraction

A primary challenge in email aggregation is “Interface Incompatibility” between disparate providers (e.g., Gmail’s REST API vs. private IMAP servers). Academic literature identifies the **Adapter Design Pattern** as a structural solution that allows incompatible interfaces to collaborate. By implementing a unified `MailAdapter` interface in NestJS, the core application remains decoupled from provider-specific logic, allowing for the seamless addition of new email services without core code modification.

2.2.2. Zero Trust and Context-Aware Security

Traditional security models often rely on a “Binary Perimeter” (authenticated vs. unauthenticated). Modern enterprise security has shifted toward **Zero Trust Architecture (ZTA)**, which posits that trust must be continuously verified. The cdcIRM “Trust Engine” applies these principles by calculating a **Trust Score** based on environmental metadata (IP Geolocation, Device Fingerprinting, and login history), ensuring that sensitive actions are only permitted when a session meets a specific trust threshold.

2.2.3. Relational Localization and Data Sovereignty

Localizing applications for government contexts (Khmer/English) requires a strategy for **Dynamic Data Sovereignty**. While static UI labels can be bundled with the client, dynamic data (such as investor categories) requires a relational approach. This project have been made from the ground up with localizing in mind.

2.2.4. High-Concurrency Data Streaming

Handling large volumes of investor correspondence requires efficient data transfer strategies:

- **WebSockets:** To replace expensive HTTP polling, WebSockets provide bi-directional, real-time updates for new emails.
- **Cursor-based Pagination:** Unlike “Offset” pagination, which can skip or duplicate records when new data is inserted, cursor-based pagination ensures a stable, high-

performance “infinite scroll” experience, which is essential for managing extensive email threads.

III. PROJECT ANALYSIS

The system analysis phase involved translating the CDC’s institutional goals into a set of technical mandates. This section outlines the functional and non-functional requirements that governed the development of the cdcIRM platform.

3.1. Requirements Specification

3.1.1. Functional Requirements

Functional requirements define the specific actions and behaviors the application must perform to satisfy user needs:

Table 1: Functional Requirements of the cdcIRM System

ID	Feature	Description
1	Email Linking	Allow the user to link their mailboxes from various email providers
2	Inbox Listing	The system shall display a list of email messages with essential metadata such as sender, subject, timestamp, and status (read/unread).
3	Investor Contact Scanning	The system shall capture an image from the device camera and extract contact data such as name, company, phone number, and email address.
4	Investor Contact Review and Edit	The system shall allow users to review and correct OCR results before saving them.
5	Investor Contact Storage	The system shall save validated contact information into the system for future communication use.
6	AI Assistance	The system shall support AI-based drafting or response suggestions for selected email messages.
7	Notification Handling	The system shall notify users about pending follow-ups, important messages, or system events.
8	Language Preference	The system shall support application localization and allow switching between Khmer and English.

3.1.2. Non-Functional Requirements

- **Multi-lingual Support:** The system must allow users to toggle between Khmer and English instantly, ensuring all system messages, data categories, and UI labels are localized.
- **Security (Contextual Trust):** The system must implement token-based authentication (JWT) and a “Trust Engine” that evaluates session metadata (IP, location, and device ID) to protect sensitive government data.
- **Performance (Cursor-Based Pagination):** To handle high-velocity email feeds without latency, the system must utilize cursor-based pagination to ensure a stable and fluid scrolling experience.
- **Scalability (Adapter Architecture):** The backend must be provider-agnostic, utilizing the Adapter Design Pattern to allow the integration of new email services without structural code changes.
- **Usability:** The interface must adhere to the provided design documentation provided by the company to ensure consistent design and facilitate the usability of CDC staffs.

Table 2: Non-Functional Requirements

Feature	Description
Usability	The user interface should be simple, readable, and optimized for mobile use by non-technical and technical staff alike. Important actions should be reachable with minimal taps.
Security	The system should use token-based authentication (JWT), secure API communication over HTTPS, and protected local storage for sensitive session data.
Reliability	The system should handle network failures gracefully by providing clear feedback and retry options without crashing.
Maintainability	The codebase should follow a modular architecture with clear separation between data models, services, state management, and UI components.
Scalability	The design should support future additions such as calendar integration, investor tagging, analytics, and advanced AI modules.
Compatibility	The mobile application should run on modern Android and iOS devices used by CDC staff.
Localization	The system should support Khmer and English text resources and be designed to allow adding more languages in the future.

3.2. Use Case Analysis

To understand the interaction between CDC officials and the cdcIRM ecosystem, the following primary use cases were identified:

3.2.1. Account Linking



Figure 3: Account Linking Use Case Diagram

Figure 3 Outlines CDC official authenticates via the CDC internal portal and links their professional Gmail or Outlook account via a secure OAuth2 handshake.

3.2.2. Scanning Business Cards

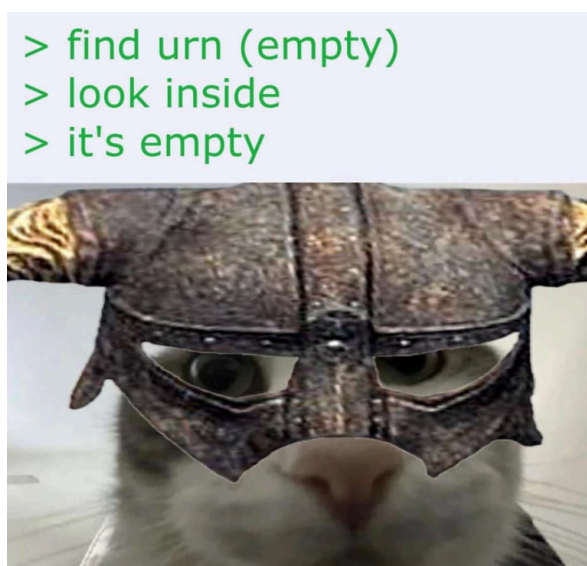


Figure 4: Business Card Scanning Use Case Diagram

Figure 4 During an investment forum, an official scans an investor's business card using the OCR module, which automatically populates the CDC's centralized investor database.

3.2.3. Primary Actor

The primary actor of the system is:

- **CDC Staff User:** Authorized officer using the mobile application for investor communication and follow-up management.

3.2.4. Supporting Actors / External Systems

- **CDC Backend API:** Provides authentication, user profile, email synchronization, and contact-related services.
- **OCR Engine:** Extracts text from captured business card or ID images.
- **Notification Service:** Delivers local or push notifications for reminders and updates.
- **AI Service:** Generates drafting suggestions and follow-up assistance.

3.2.5. Use Case Diagram (Conceptual)

The formal use case diagram can be inserted as a figure in the final report. For documentation clarity, the actor-to-use-case relationships are listed below.

- **CDC Staff User** → Login, View Inbox, Read Email, Compose Email
- **CDC Staff User** → Scan Contact, Save Contact, Create Follow-up, Receive Reminder
- **CDC Staff User** → AI Draft Suggestion
- **CDC Backend API** ↔ Authentication, Email Sync, Contact Storage
- **OCR Engine** ↔ Scan Contact
- **Notification Service** ↔ Receive Reminder
- **AI Service** ↔ AI Draft Suggestion



Figure 5: Use Case Diagram of cdcIRM (Placeholder)

3.3. Process Flow and Activity Concepts

This section describes the main activity flow of the application from login to communication and follow-up.

3.3.1. Core Workflow (Narrative)

1. The user opens the application and signs in using CDC credentials.
2. The system validates credentials through the backend and stores a secure session token.
3. The inbox is loaded and synchronized with the backend email service.
4. The user may read emails, compose new messages, or scan a contact card.
5. If scanning is used, OCR extracts text and returns candidate contact fields.
6. The user reviews and edits extracted data before saving.
7. The user can create a follow-up reminder for an investor conversation.
8. The system schedules a reminder and notifies the user at the specified time.

3.3.2. Activity Diagram (Conceptual)



Figure 6: Activity Diagram for Main User Workflow (Placeholder)

3.4. Data and Database Design

Although the internship project focuses on mobile application development, a clear data model is required to ensure smooth integration with backend APIs and future expansion. The database design presented here represents the conceptual structure of the main entities used by the system.

3.4.1. Main Entities

The system revolves around the following core entities:

- **User:** CDC staff account and profile information
- **Message:** Email metadata and content
- **Entity:** Investor or company details
- **Attachment (optional):** File metadata associated with emails
- **Audit Fields:** Created/updated timestamps for traceability

3.4.2. Conceptual Entity Relationships

- One **User** can access many **Mailboxes**
- One **User** can create many **Follow-up Reminders**
- One **Entity** can be linked to many **Email Messages**
- One **Email Message** can contain multiple **Attachments**
- One **Follow-up Reminder** may reference a **Entity** and/or an **Email Message**



Figure 7: Database Table (Placeholder)

3.4.3. Database Design Considerations

The following design principles are important for this project:

- **Normalization:** Separate contacts, emails, and reminders to avoid duplicate data.
- **Traceability:** Include timestamps for creation and updates.
- **Extensibility:** Keep optional fields flexible for future modules (e.g., investor tags, categories, priority).
- **Security:** Avoid storing sensitive tokens or secrets directly in business tables.

- **Integration-first:** The mobile app may not directly access the database, but entity design should align with backend API responses.

3.4.4. Sample Data Dictionary (Selected Fields)

Table 3: Selected Data Dictionary

Table	Field	Type	Description
user	id	UUID / String	Unique identifier of CDC staff user.
message	thread_id	String	Groups related messages into one conversation thread.
message	is_read	Boolean	Read/unread state for UI display.

3.5. Analysis of Technical Architecture

To support maintainability and future growth, the mobile application adopts a layered architecture.

3.5.1. Proposed Application Layers

- **Presentation Layer:** Flutter UI screens, widgets, and user interactions
- **State Management Layer:** Handles screen state, loading state, and user actions
- **Domain/Logic Layer:** Business rules such as validation, reminder logic, and work-flow handling
- **Data Layer:** API clients, local storage, serialization, and repository implementations
- **External Services Layer:** OCR, AI assistance, notification service, and backend APIs

This separation improves:

- Code readability
- Testing capability
- Feature scalability
- Team collaboration during future maintenance

3.5.2. API and Data Exchange Considerations

The backend communication is expected to use JSON-based REST APIs. During analysis, the following integration concerns were considered:

- Authentication token refresh and secure storage
- Standardized API response format (success/error)
- Consistent date-time parsing (ISO 8601)
- Pagination for inbox and contact lists

- Snake_case API keys mapped to camelCase mobile models
- Error handling for unstable network conditions

These considerations influenced the model and service design in the implementation phase.

IV. DETAIL CONCEPT

This chapter presents the detailed concept of the proposed **cdcIRM** system. While the previous chapter focused on analysis and requirements, this chapter translates those findings into a concrete design structure, including the logical architecture, physical architecture, and the technologies and tools required for implementation.

The purpose of this chapter is to define how the system is organized technically before implementation, so that development remains consistent, maintainable, and aligned with CDC operational needs.

4.1. Logical Architecture

The logical architecture describes the software structure of the **cdcIRM** mobile application and how its internal components interact. The system follows a layered and modular design to separate responsibilities and simplify maintenance.

4.1.1. Architecture Style

The proposed architecture uses a **layered application design** with modular features. Each layer has a specific role:

- **Presentation Layer** handles UI screens, widgets, and user interactions.
- **State Management Layer** manages UI state, asynchronous loading, and actions.
- **Domain Layer** contains business rules and workflows.
- **Data Layer** handles API communication, local storage, and model serialization.
- **Service Integration Layer** connects to OCR, notification, and optional AI services.

This design improves readability, testing, and future expansion.

4.1.2. Logical Components

Table 4: Logical Components of the cdcIRM Application

Component	Responsibility
Authentication Module	Handles login, token persistence, token refresh, logout, and access control for protected routes.
User Profile Module	Retrieves and stores authenticated user profile data such as name, role, email, and language preference.
Inbox Module	Displays email list, supports pagination, search, read/unread state, and navigation to message details.
Email Compose Module	Supports composing, replying, and forwarding messages with form validation and API submission.
Contact Scan Module	Captures image input and processes OCR results into structured contact fields.
Contact Management Module	Allows review, edit, save, and retrieve investor contact records.
Follow-up Module	Creates and manages reminders linked to contacts or email threads, including due status.
Notification Module	Schedules and displays reminder notifications for follow-up activities.
Localization Module	Provides Khmer/English language support and handles text resource loading.
Shared Core Module	Contains reusable utilities, API response wrappers, error handling, constants, and local storage services.

4.1.3. Layer Interaction Flow

The following sequence summarizes how layers interact during a common user action, such as logging in or opening the inbox:

1. The user triggers an action from the UI.
2. The presentation layer forwards the action to state management.
3. State management calls domain logic or a repository.
4. The repository accesses remote APIs or local storage.
5. The data result is mapped into app models.
6. State is updated and reflected back in the UI.

This pattern ensures that user interface code remains independent from backend communication logic.

4.1.4. Data Flow Concept

The application uses JSON-based API communication and local persistence for session-related data. Data flows through the system as follows:

- **Remote data:** backend API responses (JSON)
- **Model conversion:** snake_case JSON fields mapped to camelCase app models
- **State update:** parsed models stored in memory/state controllers
- **Local persistence:** tokens, preferences, and selected cached values stored locally
- **UI rendering:** widgets consume prepared state models

Date/time values should be converted from ISO 8601 strings into native date-time objects in the model layer to ensure consistent formatting and sorting.

4.2. Physical Architecture

The physical architecture describes the deployment environment and the hardware/software elements that host and connect the **cdcIRM** system.

The project is primarily a **mobile client application** connected to CDC-managed back-end services over a secure network. The mobile app does not directly connect to the database; instead, it communicates through APIs.

4.2.1. Deployment Overview

The physical architecture contains the following main nodes:

- **Mobile Device (Client):** Android or iOS device used by CDC staff
- **CDC Backend API Server:** Handles authentication, email operations, contacts, reminders, and tores user, contact, email metadata, and follow-up records
- **File/Attachment Storage (optional):** Stores or proxies attachment files
- **OCR Service:** Processes images for contact extraction
- **Notification Service:** Handles push or local reminder notifications
- **AI Service:** Provides smart drafting assistance



Figure 8: Physical Architecture of the cdcIRM System (Placeholder)

4.2.2. Physical Communication Flow

The high-level communication flow is:

1. User opens the mobile app and submits credentials.
2. Mobile app sends authentication request to the CDC backend API over HTTPS.
3. Backend validates credentials and returns access/refresh tokens.
4. Mobile app requests inbox, profile, contact, and follow-up data through APIs.
5. Backend reads/writes data to the database and returns JSON responses.
6. OCR and AI-related features call their respective services via backend.
7. Notification service triggers reminder alerts on the device.

4.2.3. Client Device Requirements

The mobile client should run on modern smartphones commonly used by CDC staff.

Minimum practical conditions include:

- Android or iOS device with stable internet connection
- Camera support for OCR/business card scanning
- Notification permission enabled for reminder features

4.2.4. Security Considerations in Physical Design

Because the application handles official communication and contact data, physical deployment should consider the following security measures:

- HTTPS-only API communication
- Permission-based access control
- JWT-based session authentication

- Secure local storage for tokens
- Input validation and server-side authorization checks
- Controlled access to OCR/AI services
- Logging and auditability on backend services

These controls reduce the risk of unauthorized access and data leakage.

4.3. Tools and Technology Requirements

This section defines the technologies and development tools used to design, implement, test, and maintain the **cdcIRM** internship project.

The selected technologies were chosen based on:

- compatibility with mobile development goals
- maintainability for future CDC developers
- performance and developer productivity
- support for localization and modular architecture

4.3.1. Technologies

The project combines mobile, backend integration, and service-support technologies. The table below summarizes the major technology requirements.

a. Flutter (Dart)

Flutter Logo Placeholder

Flutter is the main framework used to develop the **cdcIRM** mobile application. It supports cross-platform development, allowing the same codebase to run on both Android and iOS devices with consistent UI and behavior.

b. JWT Authentication

JWT Logo Placeholder

JWT (JSON Web Token) is used for secure authentication and session management. It enables the system to validate user access for protected API endpoints while supporting token refresh for longer user sessions.

c. Freezed and json_serializable

Freezed / JSON Serializable Logo Placeholder

These Dart packages are used to define API models with reduced boilerplate and safe JSON serialization. They improve code maintainability and ensure consistent mapping between snake_case API fields and camelCase Dart properties.

d. Local Storage / Secure Storage

Storage Logo Placeholder

Local storage is used to persist user preferences and session-related values on the device. Sensitive data such as authentication tokens should be stored using secure storage mechanisms to improve application security.

e. OCR Technology

OCR Logo Placeholder

OCR technology is used to extract contact information from business cards or ID images captured with the mobile camera. This reduces manual data entry time and improves workflow efficiency for staff handling investor contacts.

f. Notification Service

Notification Logo Placeholder

The notification service is used to deliver follow-up reminders and important alerts to users. It supports better task tracking by helping staff respond to investor communications on time.

g. Flutter Localization (I10n / ARB)

Localization Logo Placeholder

Flutter localization tools are used to support multilingual content in the app, especially Khmer and English. This ensures the interface remains accessible and appropriate for different users within the CDC environment.

4.3.2. Tools

a. Visual Studio Code / Android Studio

IDE Logo Placeholder

These development environments are used for writing, debugging, and organizing the Flutter project source code. They provide features such as code completion, terminal integration, and plugin support for Flutter development.

b. Flutter SDK

Flutter SDK Logo Placeholder

The Flutter SDK provides the framework libraries, build tools, and command-line utilities required to run and compile the mobile application. It is the core toolchain used throughout the development process.

c. Git

Git Logo Placeholder

Git is used for version control to track code changes and manage development history. It helps maintain project stability and makes collaboration easier when multiple contributors work on the same codebase.

d. GitHub / GitLab

GitHub / GitLab Logo Placeholder

A repository hosting platform is used to store and manage the project remotely. It supports source backup, collaboration workflows, and issue tracking during the internship development period.

e. Postman / Insomnia

Postman / Insomnia Logo Placeholder

API testing tools are used to verify backend endpoints before integrating them into the mobile application. They help inspect request payloads, responses, and error handling behavior during development.

f. Android Emulator / iOS Simulator / Physical Device

Device Testing Logo Placeholder

Testing environments are required to validate the application on different platforms and screen sizes. Physical devices are especially important for camera-based features such as OCR and real notification behavior.

g. Figma

Figma Logo Placeholder

Figma is used to design and preview interface layouts before implementation. It helps organize screens, user flows, and visual consistency for the mobile application during planning and refinement.

h. Typst

Typst Logo Placeholder

Typst is used to prepare this very internship project report. It offered structured formatting, figures, and chapter organization. It provides a clean workflow for maintaining technical documentation with consistent layout.

i. draw.io / Lucidchart / PlantUML

Diagram Tool Logo Placeholder

Diagramming tools are used to create architecture, use case, and activity diagrams for the report. These visual materials improve clarity and help explain system structure and workflows more effectively.

4.3.3. Development Environment Requirements

A stable development environment should include:

- Flutter SDK properly configured
- Android SDK and emulator support
- Xcode (for iOS build/testing on macOS)
- Git installed and configured
- Access to CDC backend API test environment
- Access credentials for OCR/notification services (if enabled)

These requirements ensure the application can be built and tested reliably during the internship period.

V. IMPLEMENTATION

...

5.1. Project Structures

...

5.2. Sequence Diagram

...

VI. CONCLUSION

...

6.1. Completed Functions

...

6.2. Incomplete Functions

...

6.3. Challenges

...

6.4. Experience

...

6.5. Perspective

...

6.6. Future Work

...

REFERENCES

- [1] Google LLC, "Gmail." [Online]. Available: <https://mail.google.com/>
- [2] Microsoft Corporation, "Microsoft Outlook." [Online]. Available: <https://outlook.live.com/>

APPENDICES